PES UNIVERSITY

(ESTABLISHED UNDER KARNATAKA ACT NO. 16 OF 2013)
100-ft Ring Road, Bengaluru – 560085, Karnataka, India

Capstone Project Report

on

CleanNode- IoT based Public Sanitation System

Submitted by

ADHYA PANDEY - (PES1UG24CA004)
RISHIKA CHOWDHARY.B - (PES1UG24CA025)
CHINMAYI C GOWDA - (PES1UG24CA037)
LAVANYA.R - (PES1UG24CA081)

Jan 2026 - May 2026

Under the guidance of

Mr.Dilip Kumar Maripuri
Associate Professor
Department of Computer Applications,
PES University, Bengaluru – 560085

FACULTY OF ENGINEERING

DEPARTMENT OF COMPUTER APPLICATIONS

PROGRAM - BACHELOR OF COMPUTER APPLICATIONS

Certificate

This is to certify that the project entitled

CleanNode- IoT based Public Sanitation System

is a bonafide work carried out by

ADHYA PANDEY (PES1UG24CA004)
RISHIKA CHOWDHARY.B (PES1UG24CA025)
CHINMAYI C GOWDA (PES1UG24CA037)
LAVANYA.R (PES1UG24CA081)

in partial fulfillment for the completion of **Capstone Project** work in the Program of Study BCA under rules and regulations of **PES University**, Bengaluru during the period **Jan 2026 - May 2026**. The project report has been approved as it satisfies the academic requirements of 4th semester BCA.

Guide	**Coordinator of BCA Programme**	**Chairperson**	**Dean - Faculty of Engineering & Technology**
Mr.Dilip Kumar Maripuri	**Dr. A. Lekha**	**Dr. Veena S**	
Associate Professor Dept. of Computer Applications PES University Bengaluru - 560085	Professor Dept. of Computer Applications PES University Bengaluru - 560085	Professor Dept. of Computer Applications PES University Bengaluru - 560085	PES University Bengaluru - 560085
Date :	**Date :**	**Date :**	**Date :**

DECLARATION

We, **Adhya Pandey**, bearing **PES1UG24CA004**, **Rishika Chowdhary.B**, bearing **PES1UG24CA025**, **Chinmayi C Gowda**, bearing **PES1UG24CA037**, and **Lavanya.R**, bearing **PES1UG24CA081**, hereby declare that the Capstone Project entitled *CleanNode- IoT based Public Sanitation System* is an original work done by us under the guidance of **Mr.Dilip Kumar Maripuri**, *Associate Professor*, PES University, and is being submitted in partial fulfillment of the requirements for completion of the 4th Semester course in the Program of Study BCA. All corrections and suggestions indicated for internal assessment have been incorporated in the report.

The plagiarism check has been done for the report and is below the given threshold.

We further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other course.

ADHYA PANDEY	**RISHIKA CHOWDHARY.B**	**CHINMAYI C GOWDA**	**LAVANYA.R**
(PES1UG24CA004)	**(PES1UG24CA025)**	**(PES1UG24CA037)**	**(PES1UG24CA081)**

PLACE: Bengaluru
DATE:

NAME AND SIGNATURE OF THE CANDIDATES

ACKNOWLEDGMENT

We take great pleasure in expressing our sincere gratitude to all those who have guided and supported us in the successful completion of our capstone project.

We express our heartfelt thanks to the Vice Chancellor of PES University, **Dr. J. Suryaprasad**, the Chairperson **Dr. Veena S**, and the BCA Programme Coordinator **Dr. A. Lekha** for providing us with the opportunity and necessary support to carry out this project.

We are especially grateful to our guide, **Mr.Dilip Kumar Maripuri**, Associate Professor, Department of Computer Applications, for his valuable guidance, encouragement, and continuous support throughout the project.

ADHYA PANDEY	RISHIKA CHOWDHARY.B	CHINMAYI C GOWDA	LAVANYA.R
(PES1UG24CA004)	(PES1UG24CA025)	(PES1UG24CA037)	(PES1UG24CA081)

ABSTRACT

Public toilets in Indian cities are usually cleaned on a fixed timetable, and their condition is known only when a staff member walks in to check. A facility can therefore stay unusable for hours after it becomes dirty, while one that is barely used may be cleaned several times a day. Poor upkeep in turn discourages people from using these facilities at all. Municipal bodies need a way to know the actual condition of each toilet without depending on manual inspection.

CleanNode addresses this gap with sensor nodes installed inside the restroom that report its condition continuously. Instead of following a fixed schedule, maintenance staff are alerted when readings cross set limits, so cleaning is carried out where and when it is actually needed.

Each node is built around an ESP32 microcontroller connected to an MQ-135 gas sensor for air quality, a DHT22 for temperature and humidity, a PIR sensor for occupancy, and a YF-S201 flow sensor for water usage. Readings are published over MQTT to a cloud backend, where a Random Forest model computes a Predictive Cleanliness Index (PCI) and multi-sensor rules detect conditions such as bad smell and flush failure. The results are stored in Firebase and presented on a React web dashboard for the municipal authority and an Android application for field staff and the public, through which cleaning duties can also be assigned and tracked.

The system reduces dependence on routine manual checks, shortens the time between a problem arising and staff responding to it, and allows cleaning effort to be directed to the facilities that need it most. Because it uses low-cost, readily available components, it can be deployed across a large number of public toilets at modest expense.

CONTENTS

1 Introduction **1**

1.1 Problem Definition 1

1.2 Proposed Solution 1

1.3 Purpose 1

1.4 Scope 2

2 Literature Survey **3**

2.1 Domain Survey 3

2.2 Related Work 6

2.3 Existing Systems 18

3 Hardware and Software Requirements **19**

3.1 Introduction 19

3.2 Hardware Requirements 19

3.3 Software Requirements 21

4 Software Requirements Specification **22**

4.1 Users 22

4.2 Functional Requirements 24

4.3 Non-Functional Requirements 31

5 System Design **35**

5.1 Architecture Diagram 35

5.2 Context Flow Diagram 37

6 Detailed Design **38**

6.1 Use Case Diagram 38

6.2 Hardware Integration Diagram 39

6.3 Database Design 40

6.4 Document Structure 41

7 Implementation **42**

7.1 Algorithms 42

7.2 Screenshots 44

8 Software Testing **49**

8.1 Hardware and Sensor Testing 49

8.2 Web Dashboard Testing 50

8.3 Mobile Application Testing 50

9 Conclusion **52**

10 Future Enhancements **53**

Appendix A: Bibliography **54**

LIST OF FIGURES

5.1 System Architecture of CleanNode 35
5.2 Context Flow Diagram 37

6.1 Use Case Diagram 38
6.2 CleanNode IoT Sensor Integration 39
6.3 Firebase Realtime Database Structure 40

7.1 Login Screen 44
7.2 Operations Overview 44
7.3 Facilities 46
7.4 Live Monitor 46
7.5 Anomaly Detection 47
7.6 Staff and Duties 47
7.7 Dispatch Cleaning Duty 48
7.8 Firebase Authentication 48

LIST OF TABLES

2.1 Summary of Reviewed Research Papers 17
2.2 Comparative Study of Existing Applications 18

3.1 Minimum Hardware Requirements 19
3.2 Recommended Hardware Specifications 20
3.3 Sensors Used 20
3.4 Operating System Requirements 21
3.5 Development Tools 21
3.6 Programming Languages and Frameworks 21

8.1 Subsystems Under Test 49
8.2 Test Cases: Hardware and Sensors 49
8.3 Test Cases: Web Dashboard 50
8.4 Test Cases: Mobile Application 50

1. INTRODUCTION

1.1 Problem Definition

Public toilets in cities such as Bengaluru are maintained on fixed cleaning schedules, supported by occasional manual inspection. Under this arrangement, the authorities have no direct way of knowing the condition of a facility between visits. Problems such as foul odour from poor ventilation, failure of the water supply, or heavy footfall at peak hours often go unnoticed until a user complains, and the existing complaint channels are slow to act on such reports. Users, for their part, have no means of finding out in advance whether a nearby toilet is available or in usable condition. As a result, cleaning staff are not always deployed where they are most needed, and many facilities remain in poor condition for long periods. Studies of public restrooms have also shown that poor maintenance is not only unpleasant but can expose users to harmful gases such as ammonia [1], and that facilities in poor condition discourage people from using them, undermining the purpose of public sanitation programmes [2].

1.2 Proposed Solution

CleanNode is an IoT-based monitoring system that places a sensor node inside each restroom. The node measures air quality, temperature, humidity, occupancy and water usage, and the readings are used to assess the cleanliness of the facility and to detect abnormal conditions. A web dashboard shows the live status of every facility and cubicle along with any active alerts, so that the authorities can assign a cleaning duty to a staff member as soon as a problem is detected. An Android application supports staff in the field and lets members of the public find toilets, check whether they are available and submit feedback. Cleaning is thus driven by the actual condition of each facility rather than by a fixed timetable. Readings from the different sensors are combined into a single Predictive Cleanliness Index (PCI), so that a supervisor can judge the state of a restroom from one value instead of interpreting several separate readings.

1.3 Purpose

The purpose of this project is to build a working prototype that gives maintenance authorities continuous, reliable information about the condition of public toilets. The specific

objectives are to collect sensor data on air quality, temperature, humidity, occupancy and water usage; to use this data to estimate cleanliness levels and flag abnormal conditions early; to reduce the dependence on manual inspection; and to shorten the time taken to respond to maintenance needs, so that cleaning staff and other resources are used more effectively.

1.4 Scope

The project covers public and community sanitation facilities, each of which may contain several cubicles. It includes the design of a sensor node based on the ESP32 microcontroller, the transmission of its readings over MQTT, a backend that combines rule-based logic with a machine learning model to compute the cleanliness index and detect anomalies, a web dashboard for the municipal authority, and an Android application for maintenance staff and the public. The system monitors conditions, raises alerts and supports the assignment of cleaning duties; it does not include robotic or automated cleaning, which is discussed under future enhancements.

2. LITERATURE SURVEY

2.1 Domain Survey

Introduction to the Domain

The Internet of Things (IoT) refers to physical devices fitted with sensors and network connectivity that report their readings without manual intervention. A low-cost microcontroller with a few sensors can now measure the condition of a room and send that information to a server every few seconds, where it can be stored, analysed and acted upon. Over the last decade this approach has been adopted in healthcare, agriculture, industrial automation and urban infrastructure, wherever continuous measurement is cheaper or more reliable than periodic manual checks. Cloud platforms have made it practical to manage thousands of such devices from a single place, with storage and processing that grow as more devices are added [3].

Public facilities such as toilets, waste bins and drainage lines have traditionally been maintained on fixed schedules backed by manual inspection. This method needs considerable manpower and still reacts late, because a fault is usually noticed only after it has become serious or after a citizen has complained. Sensor-based monitoring changes this by reporting the condition of a facility continuously, raising an alert as soon as a reading goes out of range, and giving administrators the data they need to plan maintenance in advance rather than respond after the fact. Such systems have already been demonstrated for restroom hygiene [1, 2], janitor scheduling [4], waste collection [5, 6] and drainage monitoring [7].

Applications of IoT to public sanitation, however, are still few and scattered. Most of the systems reported so far concentrate on one aspect of the problem, usually odour or air quality [1], or were evaluated in a single building [4], and few offer a common dashboard through which a city authority can oversee many facilities. Work in indoor air quality has shown that combining several sensors gives a more reliable picture than any one sensor alone [8], but this idea has not yet been applied to judging the cleanliness of a toilet. What

is missing is a system that measures occupancy, air quality and water usage together, combines them into an overall assessment of cleanliness, and makes this information available both to the authorities responsible for maintenance and to the public who use the facilities.

Core Terminologies and Definitions

- **Internet of Things (IoT)** – A network of physical devices embedded with sensors and connectivity that collect and exchange data over the internet automatically, without human involvement.
- **Smart Sanitation** – The use of IoT sensors, data analytics, and automated alerts to monitor and manage public sanitation facilities in real time, moving away from manual inspection-based approaches.
- **Real-Time Monitoring** – Continuous collection and transmission of sensor data to provide immediate visibility into the current condition of a facility, enabling faster response to hygiene or maintenance issues.
- **Predictive Cleanliness Index** – A composite score ranging from 0 to 100, computed by an ML model using multi-sensor inputs such as ammonia levels, occupancy count, humidity, and flush health, representing the real-time cleanliness state of a toilet facility.
- **Flush Health Monitoring** – The practice of tracking water flow activity post-occupancy to detect whether a flush occurred and whether flow rate falls within the normal range, enabling early identification of blockages or maintenance needs.
- **Usage Pattern Learning** – A data mining approach applied to historical occupancy time-series data to identify recurring traffic trends such as peak hours and day-of-week patterns, enabling proactive and schedule-based cleaning decisions.
- **Alert Notification System** – An automated mechanism that notifies the concerned authorities and staff, through the dashboard and mobile application, when sensor readings cross defined thresholds.

Associated Theoretical Concepts

- **Publish-Subscribe Model** – Each ESP32 node publishes its readings to a topic on an MQTT broker, and the backend subscribes to that topic to receive and process the data.

- **Multi-Sensor Data Fusion** – Combines readings from PIR, MQ-135, DHT22, and YF-S201 sensors to compute a unified Predictive Cleanliness Index (PCI) for each toilet facility.
- **Machine Learning for Predictive Analysis** – Used to analyze historical sensor data and occupancy trends in order to compute real-time PCI scores and generate proactive cleaning schedules.
- **Threshold-Based Alerting** – Raises an alert for BBMP supervisors and staff when sensor readings exceed predefined cleanliness or safety limits, which the authority can adjust.
- **RESTful Architecture** – Enables standardised communication over HTTP between CleanNode's backend, its React web dashboard and its Kotlin Android application.

Challenges and Open Problems

- **Scalability** – Deploying and managing hundreds of IoT sensor nodes across a city like Bengaluru generates a massive continuous stream of data, making it difficult for existing systems to maintain real-time performance without significant infrastructure overhead.
- **Real-Time Adaptability** – Most current sanitation monitoring approaches rely on fixed cleaning schedules and cannot dynamically respond to sudden changes in toilet usage or unexpected sensor anomalies.
- **Hardware Reliability** – Public toilet environments are humid, chemically exposed, and prone to vandalism, making long-term sensor reliability a persistent challenge that most research prototypes do not adequately address.

2.2 Related Work

Research Review

Ten papers were reviewed for this project. The first three deal directly with toilet and restroom monitoring [1, 2, 4]. The next group covers related urban monitoring problems, namely waste collection [5, 6], drainage [7] and public safety alerts [9], from which ideas on prediction, alerting and citizen-facing applications were drawn. The remaining papers address specific techniques used in CleanNode: multi-sensor data fusion [8], scalable cloud architecture [3] and user engagement through gamification [10]. Each paper is summarised under the headings of problem statement, proposed solution, methodology, results and conclusion.

1. **P. Deshmukh, A. Mohite, H. Bhoir, R. Patil, and A. Bhonde, "Intelligent Public Toilet Monitoring System Using IoT," in *Proc. IEEE Bangalore Humanitarian Technology Conference (B-HTC)*, Oct. 2020** [2].
   Available: IEEEXplore.

   - **Abstracted Summary**

     Manual monitoring of public toilet hygiene is inefficient and time-consuming. The proposed IoT-based system uses sensors to track odor, water usage, and occupancy, providing real-time alerts when cleaning is required. This reduces manual inspections and improves maintenance efficiency.

   - **Problem Statement**

     India lacks a centralized system to monitor public toilet cleanliness. Manual inspections waste time and manpower, leading to poor maintenance. Despite large-scale initiatives like Swachh Bharat, many facilities remain unhygienic, discouraging usage.

   - **Proposed Solution**

     The system uses a NodeMCU ESP8266 with sensors such as MQ-137 (odor), ultrasonic (usage and water level), RFID (cleaner tracking), IR (soap level), LDR (lighting), and water sensors (clog detection). Data is sent to Firebase via Wi-Fi every five minutes. A ReactJS dashboard and mobile app provide monitoring and alerts.

   - **Methodology**

     Sensor data is cleaned, normalized, and stored with timestamps. An ARIMA model forecasts daily trends, evaluated using MFE, MAE, and MAPE. RFID tracks cleaner attendance, and user feedback is collected through rating buttons installed at toilets.

   - **Results**

     The ARIMA forecasts were accurate enough to support preventive maintenance, and push notifications reduced the need for manual inspection.

   - **Conclusion**

     The system improves sanitation management through real-time monitoring and automation, although its scalability needs further work.

2. **F. I. Azman, N. L. Saleh, F. Hashim, A. Sali, A. M. Ali, and A. S. M. Noor, "An IoT-Based Hygiene Monitoring System in the Restroom,"** ***IEEE Access*****, vol. 13, pp. 119348–119361, 2025** [1].
   Available: IEEEXplore.

   - **Abstracted Summary**

     The paper presents a restroom hygiene monitoring system that triggers cleaning only when hygiene thresholds are exceeded. MQ-137 and BME680 sensors measure ammonia and indoor air quality (IAQ), and the data is sent over MQTT to a Grafana dashboard.

   - **Problem Statement**

     Fixed cleaning schedules waste labour, while poorly maintained restrooms expose users to harmful gases such as ammonia.

   - **Proposed Solution**

     The MQ-137 detects ammonia and the BME680 measures IAQ. Telegram alerts are sent when ammonia exceeds 5 ppm or IAQ exceeds 100, and an IR counter triggers cleaning after every 50 users.

   - **Methodology**

     A five-month study within a two-year dataset (2023–2024) was conducted. Statistical analysis examined relationships between temperature, humidity, and ammonia levels. Sensors were installed at optimal height in protective casing for accurate detection.

   - **Results**

     Ammonia spikes were traced to sewer problems and improper cleaning. Ammonia showed a strong negative correlation ($r = -0.91$) with temperature and humidity, and high IAQ values were linked to excessive use of cleaning chemicals.

   - **Conclusion**

     Threshold-based gas and IAQ monitoring can replace fixed schedules with condition-based cleaning. However, the system assesses only air quality: it cannot detect dirty surfaces or flush failure, and it offers no combined cleanliness score or public interface.

3. **A. Lokman, R. K. Ramasamy, and C.-Y. Ting, "Scheduling and Predictive Maintenance for Smart Toilet,"** ***IEEE Access*****, vol. 11, pp. 17983–17999, 2023** [4]. Available: IEEEXplore.

    - **Abstracted Summary**

      This paper introduces an IoT-based smart toilet system (ARIGA) that combines predictive maintenance and optimized janitor scheduling. It uses an ARIMA model to predict equipment degradation and a Genetic Algorithm to generate efficient cleaning schedules based on usage patterns.

    - **Problem Statement**

      Cleaning in high-rise buildings is costly and usually follows fixed schedules, ignoring actual usage differences across floors. This leads to inefficient staff allocation and unnecessary cleaning, as existing systems lack usage-based scheduling.

    - **Proposed Solution**

      The system uses infrared and temperature-humidity sensors with Raspberry Pi and ESP32 to collect usage data. ARIMA predicts maintenance needs, while a Genetic Algorithm optimizes janitor schedules. Simulations model user behavior and toilet usage.

    - **Methodology**

      Data was collected from university toilets and split into training and testing sets. The ARIGA model was compared with LSTM and Greedy algorithms using RMSE, MAE, and MAPE metrics. Scheduling was tested under different building sizes and user loads.

    - **Results**

      The ARIGA model outperformed LSTM with lower error rates. The scheduling algorithm reduced cleaning staff requirements by up to 24.7% and improved efficiency by scheduling tasks during off-peak hours.

    - **Conclusion**

      The system enhances maintenance planning through predictive analytics and optimized scheduling. It reduces costs and downtime, though further work is needed to improve prediction accuracy.

4. **T. Stephan, S. M. Hari Krishna, C.-C. Lin, U. Sumesh, S. Agarwal, and H. Kim, "ProWaste for Proactive Urban Waste Management Using IoT and Machine Learning," *Scientific Reports*, vol. 15, Jul. 2025** [5].
   Available: Springer Nature (Scientific Reports).

   - **Abstracted Summary**

     This paper presents ProWaste, an IoT and machine learning platform for proactive waste management. It predicts the criticality of waste collection centers using multiple data sources and sends alerts to maintenance teams via a mobile app, improving efficiency and responsiveness.

   - **Problem Statement**

     Waste collection systems are often reactive, leading to overflow issues. Existing models overlook key factors like location and weather, and few provide complete real-world implementations for efficient waste management.

   - **Proposed Solution**

     ProWaste integrates sensor data, public APIs, and mobile inputs into a Firebase database. Fifteen features such as population density, weather, and waste trends are used in a machine learning model. A Flutter app alerts teams and dynamically prioritizes collection tasks.

   - **Methodology**

     Twenty-five classifiers were tested using cross-validation. Binary Particle Swarm Optimization reduced features by 80%. The system was evaluated using 6,954 records from 57 centers over four months to identify key influencing factors.

   - **Results**

     A Decision Tree model achieved over 99% accuracy with only three features. The system showed reliable real-time performance and reduced unnecessary waste collection operations.

   - **Conclusion**

     ProWaste improves waste management through predictive analytics, reducing costs and enhancing service quality. It is scalable for smart cities, with scope for further real-time data integration.

5. **Q. P. Ha, S. Metia, and M. D. Phung, "Sensing Data Fusion for Enhanced Indoor Air Quality Monitoring," *IEEE Sensors Journal*, vol. 20, no. 8, pp. 4430–4441, Apr. 2020** [8].
   Available: IEEEXplore.

   - **Abstracted Summary**

     This paper proposes a multi-sensor data fusion approach for indoor air quality monitoring. It combines IAQI and humidex into an Enhanced Indoor Air Quality Index (EIAQI) using an Extended Fractional-Order Kalman Filter (EFKF) for accurate real-time predictions.

   - **Problem Statement**

     Indoor air quality depends on multiple pollutants and environmental factors. Individual sensors often produce noisy or incomplete data, making single-sensor monitoring unreliable for accurate assessment.

   - **Proposed Solution**

     The system uses Waspmote sensors to measure multiple pollutants along with temperature and humidity. IAQI and humidex values are computed and fused into EIAQI using a weighted model with EFKF to handle noise, nonlinearity, and missing data.

   - **Methodology**

     Deployed in a university building with over 100 sensors, the system used EFKF with spatial modelling. Performance was evaluated using MAPE, RMSE, and R^2 metrics, including analysis of a real incident case.

   - **Results**

     The system achieved low error rates (MAPE: 0.3–0.9). The fused EIAQI effectively identified poor air quality conditions, proving more reliable than individual sensor readings.

   - **Conclusion**

     Data fusion significantly improves air quality monitoring accuracy and reliability. The approach supports real-time assessment, though further optimization can enhance efficiency.

6. **M. Vibhute, J. Biju, K. Shelar, N. Koli, and O. Gubyad, "Smart Drainage Monitoring System,"** ***International Journal for Multidisciplinary Research*****, vol. 7, no. 2, Apr. 2025** [7].
   Available: IJFMR.

   - **Abstracted Summary**

     This paper presents a smart drainage monitoring system using flow, temperature, and gas sensors to detect pipeline anomalies. The system uses an Arduino with GSM-based SMS alerts for real-time blockage detection and monitoring.

   - **Problem Statement**

     Drainage blockages can cause waterlogging, damage, and environmental risks. Traditional monitoring methods rely on manual inspection, which is inefficient and often fails to detect issues early.

   - **Proposed Solution**

     The system uses a YF-S201 flow sensor to detect blockages via reduced flow rate, a DS18B20 sensor for temperature monitoring, and an MQ-135 gas sensor for harmful gases. An Arduino processes data, displays it on an LCD, and sends alerts via a GSM module.

   - **Methodology**

     The system operates using a Hall-effect flow sensor to measure fluid movement. Data is transmitted to the ThingSpeak cloud using GPRS for remote monitoring. It is designed for use in drainage systems, pipelines, and flood control.

   - **Results**

     The system successfully detected blockages, temperature anomalies, and gas presence in real time. SMS alerts were triggered promptly, proving the system effective and scalable.

   - **Conclusion**

     The solution enables early detection of drainage issues, reducing manual effort and preventing infrastructure damage. It is cost-effective and suitable for urban applications, with potential for large-scale deployment.

7. **H. Zhang, R. Zhang, and J. Sun, "Developing Real-Time IoT-Based Public Safety Alert and Emergency Response Systems," *Scientific Reports*, vol. 15, Aug. 2025** [9].

   Available: Springer Nature (Scientific Reports).

   - **Abstracted Summary**

     This paper presents a real-time IoT-based system for public safety and emergency response. It integrates sensors, edge computing, and cloud platforms to detect, classify, and deliver alerts quickly during critical incidents.

   - **Problem Statement**

     Existing safety systems rely on manual reporting and outdated communication methods, making them slow and reactive. They lack real-time data integration and coordinated response capabilities.

   - **Proposed Solution**

     The system uses multiple sensors (gas, flame, vibration, biometric) with edge devices like Raspberry Pi and ESP32. Data is transmitted securely via MQTT, with LoRa as backup. Alerts are delivered through dashboards, mobile notifications, and SMS.

   - **Methodology**

     A prototype was tested in four scenarios: fire, accidents, gas leaks, and medical emergencies. Performance was evaluated based on latency, accuracy, scalability, and reliability, and compared with existing systems.

   - **Results**

     The system achieved alert latency under 450 ms, detection accuracy exceeding 95%, scalability supporting over 12,000 concurrent devices, 99.1% alert success rate, and 99.8% uptime. The comparison against existing systems confirmed superior performance in latency, reliability, and scalability.

   - **Conclusion**

     The work shows that combining edge processing with cloud delivery can bring alert latency well below a second while supporting thousands of devices. The authors identify predictive analytics as the next step, so that incidents can be anticipated rather than only detected.

8. **K. Dineva and T. Atanasova, "Design of Scalable IoT Architecture Based on AWS for Smart Livestock,"** ***Animals*****, vol. 11, no. 9, Art. no. 2697, Sep. 2021** [3]. Available: MDPI.

    - **Abstracted Summary**

      This paper presents a scalable IoT architecture built on AWS for livestock monitoring. It uses cloud-based services for data collection, processing, and analytics, ensuring automation, security, and efficient system performance.

    - **Problem Statement**

      As the number of connected devices grows, existing multi-layer and microservice architectures struggle to handle large volumes of heterogeneous data, and most livestock systems lack central device management, remote configuration and security updates.

    - **Proposed Solution**

      The architecture is built entirely on AWS, using IoT Core for device connectivity, streaming services for ingestion, a data lake for storage, machine learning services for analytics and notification services for alerts, following the AWS Well-Architected Framework.

    - **Methodology**

      The system was developed using Agile (Scrum) and stress-tested for data processing, latency and reliability under high load.

    - **Results**

      The architecture processed the required data volume, scaled automatically and handled up to 10,000 IoT devices per second with low latency.

    - **Conclusion**

      Managed cloud services can absorb the data load of a large IoT deployment and scale as devices are added, which is equally relevant to a city-wide network of toilet sensors.

9. **K. Pardini, J. J. P. C. Rodrigues, O. Diallo, A. K. Das, V. H. C. de Albuquerque, and S. A. Kozlov, "A Smart Waste Management Solution Geared Towards Citizens," *Sensors*, vol. 20, no. 8, Art. no. 2380, Apr. 2020** [6].
   Available: MDPI.

   - **Abstracted Summary**

     The paper proposes a citizen-centric smart waste management solution comprising a prototype smart container, web and mobile applications, and an IoT middleware platform that connects them.

   - **Problem Statement**

     Municipal waste collection usually follows fixed routes and timetables, so trucks visit bins that are nearly empty while others overflow. Citizens, on their part, have no information about the status of nearby bins and play little active role in keeping their surroundings clean.

   - **Proposed Solution**

     The authors built a smart container that reports its fill level, connected it to the In.IoT middleware, and developed a web application for the collection service and a mobile application for citizens.

   - **Methodology**

     The system follows a three-layer IoT architecture of perception, network and application layers. The prototype was deployed and validated in a real-scale experiment, with the In.IoT middleware providing storage, processing and APIs.

   - **Results**

     The experiment showed that citizens could see the status of bins in real time and that route optimisation reduced unnecessary truck trips, saving economic and material resources.

   - **Conclusion**

     The study shows that sharing sensor data with citizens, and not only with the collection service, improves both participation and the use of collection resources. This citizen-facing design is directly relevant to a public toilet monitoring system.

10. **M. Romano, P. Díaz, and I. Aedo, "Gamification-less: May Gamification Really Foster Civic Participation? A Controlled Field Experiment,"** ***Journal of Ambient Intelligence and Humanized Computing*****, Jun. 2021** [10].
Available: Springer.

- **Abstracted Summary**

  The paper reports a between-group experiment that compares a gamified and a non-gamified mobile application to test whether gamification increases civic participation, using Self-Determination Theory as its basis.

- **Problem Statement**

  Civic participation platforms in smart communities often suffer from low engagement, and motivation to take part must be actively encouraged through their design.

- **Proposed Solution**

  The gamified application, iWarn, uses points, rankings and levels linked to competence, relatedness and autonomy; a non-gamified version with the same functions served as the control.

- **Methodology**

  Twenty participants were divided into two groups for one week, and engagement was measured through questionnaires and usage data such as logins, missions and uploads.

- **Results**

  The gamified group logged in more often and completed more missions and uploads than the control group.

- **Conclusion**

  The experiment suggests that game elements, when tied to users' sense of competence and belonging, can raise participation in civic applications. The sample was small and the study lasted only one week, so the long-term effect remains to be established.

Summary of Reviewed Research Papers

Table 2.1: Summary of Reviewed Research Papers

Paper	Authors & Year	Problem Statement	Methodology	Key Findings
Intelligent Public Toilet Monitoring System Using IoT [2]	P. Deshmukh,Abhishek Mohite,Harshal Bhoir,Rushikesh Patil,Aparna Bhonde, 2020	Manual monitoring is time-consuming and inefficient	IoT sensors with real-time monitoring and prediction	Improves hygiene and reduces manual effort
IoT-Based Hygiene Monitoring System in Restroom [1]	F. I. Azman, N. L. Saleh, F. Hashim, 2025	Fixed cleaning schedules lead to poor hygiene	Sensor-based monitoring with data analysis	Enables condition-based cleaning
Scheduling and Predictive Maintenance for Smart Toilet [4]	A. Lokman, R. K. Ramasamy, C.-Y. Ting, 2023	Lack of usage-based scheduling increases cost	Predictive models with scheduling techniques	Improves planning and reduces cost
ProWaste: Urban Waste Management Using IoT and ML [5]	T. Stephan,S.M.Hari Krishna,Chia-Chen Lin,Ujjwal Sumesh,Saurabh Agarwal and Kim Hyunsung, 2025	Reactive waste collection leads to overflow issues	IoT with machine learning techniques	Improves waste collection efficiency
Indoor Air Quality Monitoring using Sensor Fusion [8]	Q. P. Ha,S. Metia, and M. D. Phung, 2020	Single sensor data is unreliable	Multi-sensor data fusion techniques	Improves monitoring accuracy
Smart Drainage Monitoring System [7]	M. Vibhute et al., 2025	Manual inspection fails to detect blockages	Sensor-based monitoring with alert system	Enables early detection
IoT-Based Emergency Alert System [9]	H. Zhang, R. Zhang, J. Sun, 2025	Delay in emergency response	Real-time monitoring with alerts	Improves response time
AWS-Based IoT Architecture for Livestock [3]	Kristina Dineva and Tatiana Atanasova, 2021	Existing systems lack scalability	Cloud-based IoT architecture	Supports large-scale systems
Smart Waste Management Solution [6]	Kellow Pardini,Joel J.P.C Rodrigues, 2020	Traditional waste collection is inefficient	IoT monitoring with mobile applications	Optimizes resource usage
Gamification for Civic Participation [10]	Marco Romano, Paloma Díaz, Ignacio Aedo, 2021	Low user engagement in systems	Gamification-based application approach	Increases user participation

Identified Research Gaps

- **No Flush Health Monitoring** – None of the reviewed toilet monitoring systems checks whether the flush actually operates after use, even though a failed flush is one of the most common reasons a toilet becomes unusable [1, 2, 4].
- **No Composite Cleanliness Score** – Existing systems report ammonia, humidity or temperature as separate values. Without a single combined score, a supervisor must interpret several readings to judge whether a restroom needs attention [1, 8].

- **Lack of Usage-Based Scheduling** – Apart from the work of Lokman et al., the systems reviewed either keep fixed cleaning schedules or react only after conditions have worsened; live usage data is rarely used to plan cleaning in advance [4].
- **Limited Scalability** – Most sanitation prototypes were tested in a single building or a handful of restrooms, and their behaviour across a city-wide network of facilities has not been evaluated [3].
- **No Emergency Alert Mechanism** – Emergency alerting has been studied for public safety in general, but the reviewed sanitation systems do not raise urgent alerts for conditions such as gas build-up, flooding or unusually long occupancy [9].
- **Lack of User Engagement Techniques** – Gamification has been shown to raise participation in civic applications, yet none of the sanitation systems reviewed uses it to encourage users to report problems or rate facilities [10].
- **No Citizen-Facing Application** – The reviewed toilet monitoring systems are designed mainly for authorities and maintenance staff. Apart from on-site rating buttons, they give the public no way to find a nearby clean restroom or report a problem remotely [2, 6].

2.3 Existing Systems

Table 2.2: Comparative Study of Existing Applications

Application	Features	Performance	User Experience	Limitations
Swachhata-MoHUA App	Complaint-based reporting, GPS tagging, photo upload	Works well for large-scale issue reporting	Simple and easy for citizens to use	Reactive system, no automatic monitoring or verification
Indore Smart Sanitation Dashboard	IoT sensors, real-time alerts, AI-based prediction	High efficiency in monitoring multiple toilets	Dashboard-based monitoring for authorities	Expensive, proprietary, not easily scalable
NAMMA TOILET	Multi-sensor monitoring, RFID tracking, data analytics	Conceptually strong with multiple features	Limited real-world usage	Not deployed in real environment
IAQ-Based Monitoring Systems	Gas sensors, air quality detection	Detects odor and pollution effectively	Basic alert-based interaction	Cannot measure actual cleanliness
Predictive Cleaning Systems	Usage-based prediction, scheduling algorithms	Improves cleaning efficiency	Limited user interaction	No real-time monitoring integration
Citizen Feedback Apps	Issue reporting, feedback system	Useful for complaint tracking	Easy for users to report problems	No automation, depends on user input

3. HARDWARE AND SOFTWARE REQUIREMENTS

3.1 Introduction

This chapter lists the hardware and software requirements for developing and running CleanNode. Components were chosen for their performance, scalability, cost-effectiveness, and compatibility with one another, since the sensor node, backend and application must exchange data continuously. The system combines sensors, microcontrollers, a backend server, a web dashboard and an Android application.

3.2 Hardware Requirements

The development machine must be able to run the Arduino IDE, the backend server, the MQTT broker and the Android emulator at the same time, along with the sensor node hardware itself.

Minimum Hardware Specifications

The following table lists the minimum hardware configuration required:

Table 3.1: Minimum Hardware Requirements

Component	Specification
Processor	Intel Core i5 or equivalent
RAM	4GB or more
Storage	64GB or more
Microcontroller	ESP32 DevKit v1
Sensors	MQ-135, DHT22, PIR, Water Flow Sensor (YF-S201)

Recommended Hardware Specifications

For optimal performance, the recommended configuration is:

Table 3.2: Recommended Hardware Specifications

Component	Specification
Processor	Intel Core i5 / Ryzen 5
RAM	8GB or higher
Storage	128GB or higher
Microcontroller	ESP32 DevKit v1
Sensors	MQ-135, DHT22, PIR, Water Flow Sensor (YF-S201)
Power Supply	5V 2A Adapter
Additional Components	10 kΩ resistor, push buttons, connecting wires

Sensors

The sensor node uses the following components to measure environmental conditions, occupancy and water usage.

Table 3.3: Sensors Used

Sensors	Description
ESP32 DevKit v1	Microcontroller with built-in Wi-Fi, operates at 3.3V
MQ-135 Gas Sensor	Detects ammonia and other gases that cause bad odour
DHT22 Sensor	Measures temperature (-40°C to 80°C) and humidity
PIR Sensor (HC-SR501)	Detects motion within 5–7 meters range
Push Buttons / Switches	Simulate sensor and status inputs during testing
Water Flow Sensor (YF-S201)	Measures water flow and detects flush activity

3.3 Software Requirements

The software used in the project falls into three groups: development tools, programming languages, and frameworks.

Operating System

The project was developed and tested on the following operating systems.

Table 3.4: Operating System Requirements

Operating System	Version
Windows	Windows 10/11 (64-bit)
Linux	Ubuntu 20.04+ / Fedora 36+
macOS (Optional)	macOS 11+

Development Tools

The following tools were used for coding, debugging and testing:

Table 3.5: Development Tools

Software	Purpose
Arduino IDE v2.3.2	Programming ESP32
Visual Studio Code	Code editor / IDE
Android Studio Panda 4 (2025.3.4)	Android application development
Firebase Console	Database and authentication management
Git & GitHub/GitLab	Version control & collaboration
Postman v11.89	API Testing

Programming Languages & Frameworks

The languages, frameworks and services used in each part of the system are listed below:

Table 3.6: Programming Languages and Frameworks

Technology	Usage
C/C++ (Arduino)	ESP32 firmware
Python v3.12 + FastAPI	Backend REST API
Scikit-Learn (RandomForestRegressor)	Cleanliness index (PCI) model
Pandas v2.0+, NumPy v1.24+	Data processing
MQTT v3.1.1 (HiveMQ Cloud)	Sensor data communication
Firebase Realtime Database	Data storage
Firebase Authentication	User login and roles
React 18, Vite, Tailwind CSS	Web dashboard
Kotlin, Jetpack Compose (Android 14)	Android application
Android Room (SQLite), CameraX	Local storage and camera in the app

4. SOFTWARE REQUIREMENTS SPECIFICATION

4.1 Users

- **Municipal Authority (Administrator)**

  The municipal authority is responsible for the upkeep of public toilets in its area. Using CleanNode, its officials can view live readings of air quality, occupancy and water usage for each facility and identify problems without visiting the site.

  The dashboard shows the cleanliness status of every facility and highlights any abnormal condition, which helps the authority decide where to send staff and how to allocate resources. In Bengaluru, this role would be performed by a body such as BBMP.

  Maintenance Staff

  Maintenance staff clean and maintain the restrooms. Instead of following a fixed timetable, they receive an alert from the system when a facility needs cleaning or when a fault is detected.

  Because each alert states the facility and the nature of the problem, staff can respond more quickly and avoid visits to restrooms that do not need attention.

Public

Members of the public are the people who use the restrooms. They benefit directly from facilities that are cleaned when needed rather than on a fixed schedule.

The system can also show them the cleanliness status and availability of nearby facilities, so that they can choose a usable toilet and are more willing to use public facilities.

4.2 Functional Requirements

FR1. **Cleanliness Status Evaluation and Scoring**

Field	Details
Description	The system shall analyse multiple sensor inputs and compute a cleanliness score to determine the overall hygiene level of the restroom.
Actors	Municipal Authority
Inputs	Sensor readings (air quality, humidity, occupancy, water usage)
Processing	Combine multiple sensor parameters, calculate cleanliness score, evaluate hygiene level
Outputs	Cleanliness score and status (Clean / Moderate / Dirty)
Preconditions	Valid sensor data must be available
Postconditions	Cleanliness level is updated

FR2. **Real-Time Monitoring of Restroom Conditions**

Field	Details
Description	The system shall continuously collect real-time data from sensors to monitor restroom conditions.
Actors	Municipal Authority
Inputs	Sensor data (air quality, temperature, humidity, occupancy, water flow)
Processing	Collect sensor data, transmit to database, update system in real-time
Outputs	Live data displayed on dashboard
Preconditions	Sensors must be installed and connected
Postconditions	Updated data available for monitoring

FR3. **Alert and Notification System**

Field	Details
Description	The system shall generate alerts when abnormal or critical conditions are detected.
Actors	Municipal Authority, Maintenance Staff
Inputs	Sensor threshold values, gas levels, water failure, long occupancy
Processing	Compare data with predefined limits, detect abnormal conditions, trigger alerts
Outputs	Alert notifications
Preconditions	Threshold values must be defined
Postconditions	Alerts generated for action

FR4. **Occupancy Detection**

Field	Details
Description	The system shall detect user presence and analyse restroom usage patterns.
Actors	Municipal Authority
Inputs	PIR sensor data
Processing	Detect movement, track usage, analyse usage patterns
Outputs	Occupancy and usage data
Preconditions	Sensors must be active
Postconditions	Usage data updated

FR5. **Water Usage Monitoring and Flush Detection**

Field	Details
Description	The system shall monitor water flow, detect flush activity, and identify improper flushing.
Actors	Municipal Authority
Inputs	Water flow sensor data
Processing	Analyse flow rate, detect flush activity, identify abnormal conditions
Outputs	Water usage status
Preconditions	Water flow sensor must be functioning
Postconditions	Water usage data updated

FR6. **Usage Pattern Learning and Predictive Scheduling**

Field	Details
Description	Analyzes historical occupancy data to identify peak usage patterns and generate proactive cleaning schedules.
Actors	Municipal Authority, Maintenance Staff
Inputs	Historical occupancy data, day/time metadata
Processing	Detects peak usage trends and generates cleaning schedules accordingly
Outputs	Predicted peak periods and recommended cleaning schedule
Preconditions	At least 7 days of stored occupancy data required
Postconditions	Updated cleaning schedule available to maintenance staff

FR7. **Smart Routing for Maintenance Staff**

Field	Details
Description	The system shall generate an optimized cleaning route when multiple restrooms require maintenance.
Actors	Municipal Authority, Maintenance Staff
Inputs	List of restrooms requiring cleaning, PCI scores, geographic coordinates, current location of maintenance staff
Processing	Identify restrooms needing cleaning and generate shortest route
Outputs	Optimized route displayed to staff
Preconditions	Multiple restrooms need cleaning, staff available
Postconditions	Route assigned and recorded

FR8. **Dashboard and Data Visualization**

Field	Details
Description	The system shall provide a web-based dashboard to display system data and alerts.
Actors	Municipal Authority
Inputs	Processed system data
Processing	Format and visualize data
Outputs	Dashboard with real-time information
Preconditions	System must be connected to server
Postconditions	Data is visible to users

FR9. **Maintenance History Log**

Field	Details
Description	The system shall maintain a complete timestamped log of all cleaning events, alerts, sensor readings, flush activity, and maintenance visits per restroom unit for audit and analysis purposes.
Actors	Municipal Authority
Inputs	Sensor readings, alert records, cleaning event records, flush logs, staff visit records
Processing	Aggregate all system events per restroom unit with timestamps, store in structured log, provide filterable view by date, restroom, event type, or staff member
Outputs	Maintenance history report per restroom unit; exportable log for BBMP audit purposes
Preconditions	System must be operational; all FRs generating events must be active
Postconditions	Log updated after every system event; historical data retained for minimum 90 days

FR10. **Mobile App functionality**

Field	Details
Description	The mobile app enables authorities to monitor restroom conditions, manage maintenance remotely, while allowing users to locate restrooms, check their status via a map.
Actors	Municipal authority, Public, Maintenance Staff.
Inputs	Sensor insights (ammonia levels, humidity, temperature, water flow), occupancy status (PIR), system alerts, user login credentials
Processing	Fetch real-time insights from cloud database, analyze sensor thresholds, trigger notifications, display dashboard insights, and allow user interaction for task updates
Outputs	Real-time dashboard display; push notifications for alerts; maintenance task updates; historical data visualization, heat map.
Preconditions	Mobile application must be installed; internet connectivity required; user must be authenticated; IoT system must be active
Postconditions	Alerts are viewed and acted upon; maintenance tasks updated; system logs user actions and updates in database.

FR11. **Heatmap Visualization of Restroom Conditions**

Field	Details
Description	Displays a real-time city-level heatmap of restroom cleanliness using PCI-based color coding.
Actors	Municipal Authority
Inputs	PCI scores and location coordinates of all registered restroom units
Processing	Maps PCI scores to location data and renders a color-coded heatmap (clean, moderate, dirty).
Outputs	Interactive heatmap displayed on BBMP dashboard with drill-down capability per restroom unit
Preconditions	PCI scores must be computed for all units; location data must be registered
Postconditions	Heatmap is updated in real time as PCI scores change

FR12. **Nearest Clean Restroom Navigation**

Field	Details
Description	Provides map-based navigation to the nearest clean restroom using location and real-time PCI.
Actors	Public
Inputs	User location, nearby restroom PCI scores, coordinates
Processing	Finds nearby restrooms, filters by cleanliness, ranks by distance, and shows directions
Outputs	Map with nearest clean restrooms, distance, status, and navigation
Preconditions	Location permission enabled
Postconditions	User navigated to restroom; access logged

FR13. **Multi-Facility and Cubicle Management**

Field	Details
Description	The system shall manage several facilities, each organised into blocks and cubicles, and monitor every cubicle separately.
Actors	Municipal Authority, Supervisor
Inputs	Facility details, block and cubicle identifiers, sensor node mapping
Processing	Register facilities and cubicles, link each sensor node to its cubicle, group readings by facility
Outputs	Facility list with PCI, cubicle count, alerts and occupancy for each facility
Preconditions	Sensor nodes installed and mapped to cubicles
Postconditions	Facility and cubicle records available for monitoring

FR14. **Multi-Sensor Anomaly Detection**

Field	Details
Description	The system shall detect unusual combinations of sensor conditions and assign each a severity level.
Actors	Municipal Authority, Maintenance Staff
Inputs	Gas level, humidity, occupancy, water flow and PCI for each cubicle
Processing	Check for high gas, abnormal humidity, high occupancy, occupancy without water flow and low PCI; combine the conditions found; assign severity
Outputs	List of anomalies with severity Critical, Moderate or Minor
Preconditions	Live sensor data and PCI available
Postconditions	Anomalies displayed and stored for review

FR15. **Cleaning Duty Assignment and Reassignment**

Field	Details
Description	The system shall allow the authority to assign a cleaning duty to a staff member and reassign it if the staff member cannot complete it.
Actors	Municipal Authority, Supervisor, Maintenance Staff
Inputs	Facility, cubicle, staff member, priority, remarks
Processing	Create the duty, notify the staff member, track its status, and allow it to be moved to another available staff member
Outputs	Duty record with assigned staff, priority and status
Preconditions	Staff profiles available; an alert or request for cleaning exists
Postconditions	Duty recorded in assignments; staff member notified; status updated on completion

4.3 Non-Functional Requirements

NFR1. **Performance**

Field	Details
Description	System shall provide real-time monitoring with minimal delay for timely issue detection.
Constraints	• Stable network connectivity required • Sensors must respond within acceptable latency
Acceptance Criteria	• Data refresh $\leq$ 5 seconds • Alerts generated within 2 seconds of event detection
Metrics	• Latency (ms) • Throughput (events/sec)
Priority	High

NFR2. **Reliability**

Field	Details
Description	The system shall operate consistently and provide accurate data without frequent failures.
Constraints	• Sensors must be properly calibrated • Hardware components should be maintained
Acceptance Criteria	• System should run continuously without crashes • Data accuracy should be maintained
Metrics	• System uptime • Failure rate
Priority	High

NFR3. **Scalability**

Field	Details
Description	The system shall support expansion to multiple restroom units and locations.
Constraints	• Backend system must handle increased data load • Network infrastructure should support scaling
Acceptance Criteria	• System should handle multiple devices without performance drop • Data from multiple locations should be managed efficiently
Metrics	• Number of supported devices • Response time under load
Priority	High

NFR4. **Usability**

Field	Details
Description	System shall provide a simple and user-friendly interface for easy monitoring.
Constraints	• Interface design should be clear and minimal • Navigation should be easy
Acceptance Criteria	• Users should understand dashboard without training • Information should be clearly displayed
Metrics	• User satisfaction score • Task completion time
Priority	Medium

NFR5. **Security**

Field	Details
Description	System shall ensure secure access and data protection.
Constraints	• Only authorized users should access the system • Data transmission should be secure
Acceptance Criteria	• Unauthorized access should be prevented • Data should remain protected
Metrics	• Number of security incidents • Authentication success rate
Priority	High

NFR6. **Maintainability**

Field	Details
Description	The system shall be modular so that each component can be updated without affecting the others.
Constraints	• Hardware, backend and interfaces kept as separate modules • Thresholds configurable without code changes
Acceptance Criteria	• A module can be updated independently • Limits can be changed from the Settings page
Metrics	• Time taken to apply a change • Number of modules affected by a change
Priority	Medium

NFR7. **Availability**

Field	Details
Description	The system shall remain accessible for monitoring and alerts at all times.
Constraints	• Depends on cloud services and network connectivity • Sensor nodes require continuous power
Acceptance Criteria	• Dashboard and alerts reachable whenever needed • Service resumes automatically after a network outage
Metrics	• Service uptime (%) • Time to recover after an outage
Priority	High

5. SYSTEM DESIGN

5.1 Architecture Diagram

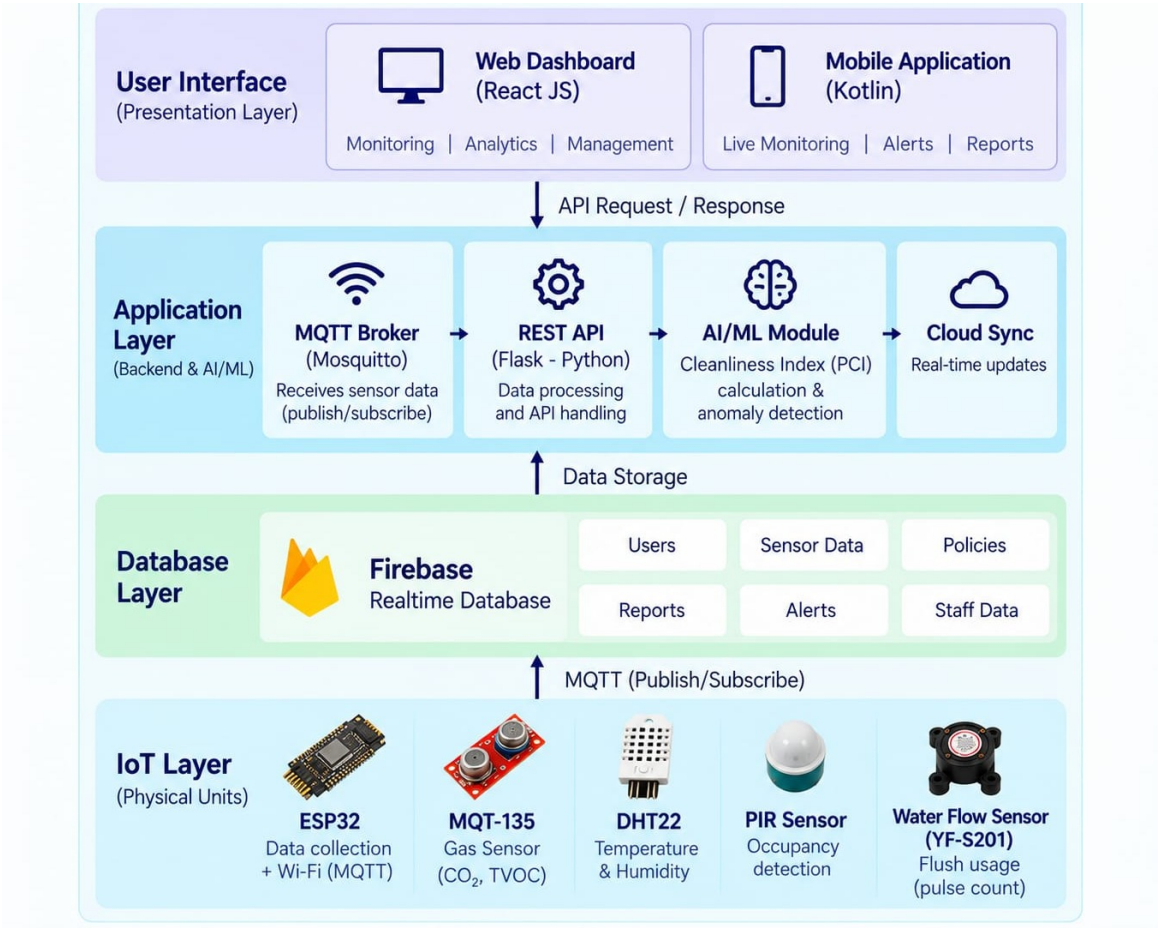


Fig. 5.1: System Architecture of CleanNode

CleanNode is organised into four layers, each of which can be changed or scaled without affecting the others. Fig. 5.1 shows these layers and the direction in which data moves between them.

The IoT Layer consists of the sensor node installed in each cubicle. An ESP32 microcontroller reads an MQ-135 gas sensor for air quality, a DHT22 sensor for temperature and humidity, a PIR sensor for occupancy and a YF-S201 flow sensor for water usage, and publishes the readings over Wi-Fi using the MQTT protocol.

The Database Layer is the Firebase Realtime Database. It holds the live sensor readings together with the records the rest of the system depends on: users, alerts, staff data, reports and policies such as the cleanliness thresholds.

The Application Layer contains the backend services. The MQTT broker receives mes-

sages from the sensor nodes, the REST API handles requests from the user interfaces, the AI/ML module computes the cleanliness index (PCI) and detects anomalies, and the cloud sync service pushes updates to the interfaces in real time.

The User Interface Layer provides two front ends. The web dashboard, built with React, is used by the municipal authority for monitoring, analytics and management. The Android mobile application, built with Kotlin, is used for live monitoring in the field, for receiving alerts and by members of the public.

5.2 Context Flow Diagram

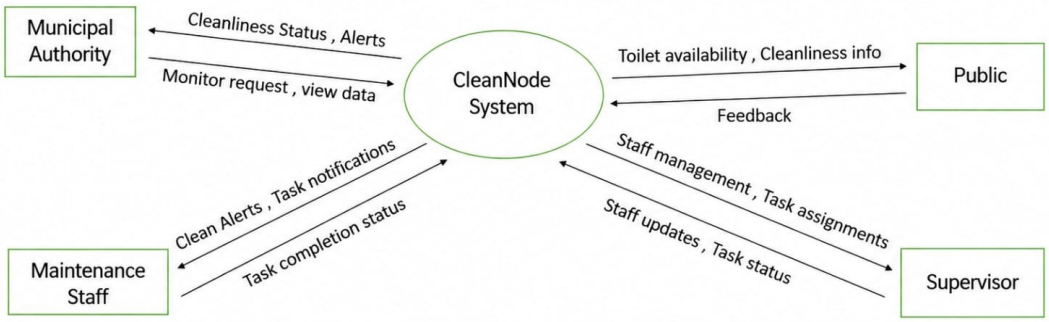


Fig. 5.2: Context Flow Diagram

The context flow diagram in Fig. 5.2 treats CleanNode as a single process and shows the four external entities that exchange information with it.

The Municipal Authority sends monitoring requests and requests to view data, and receives the cleanliness status of each facility along with any alerts. Members of the Public receive information on toilet availability and cleanliness and can send feedback to the system. Maintenance Staff receive cleaning alerts and task notifications, and report the completion status of each task. The Supervisor manages staff and assigns tasks through the system, and receives staff updates and the status of each task in return.

Within the system, data passes through a fixed sequence. The sensors collect readings and the readings are stored in the database. The AI/ML module analyses each reading to assess cleanliness, and the results are written back to the database. Where a reading or result is abnormal, the system raises an alert, and the dashboard and mobile application display the current results to their users.

6. DETAILED DESIGN

6.1 Use Case Diagram

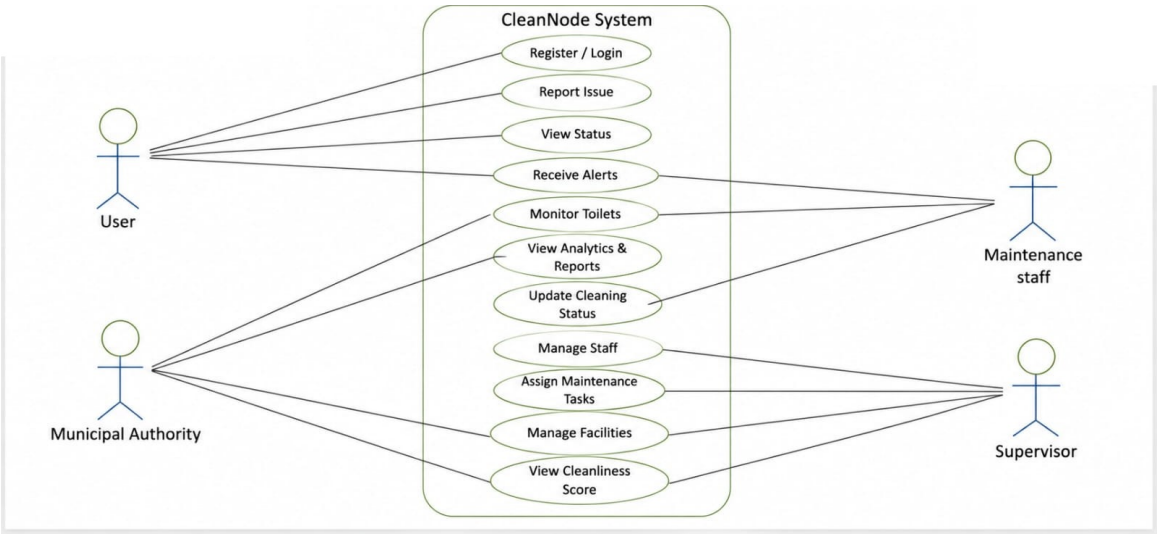


Fig. 6.1: Use Case Diagram

Fig. 6.1 shows the four actors of CleanNode and the use cases available to each of them.

A User, that is, a member of the public, can register or log in, report an issue at a facility, view the status of nearby toilets and receive alerts. The Municipal Authority monitors the toilets under its control, views analytics and reports, manages facilities and views the cleanliness score of each facility. Maintenance Staff receive alerts, monitor the toilets assigned to them and update the cleaning status once a task is complete. The Supervisor manages the maintenance staff, assigns maintenance tasks, manages facilities and views cleanliness scores, so that work can be directed to the facilities that need it.

6.2 Hardware Integration Diagram

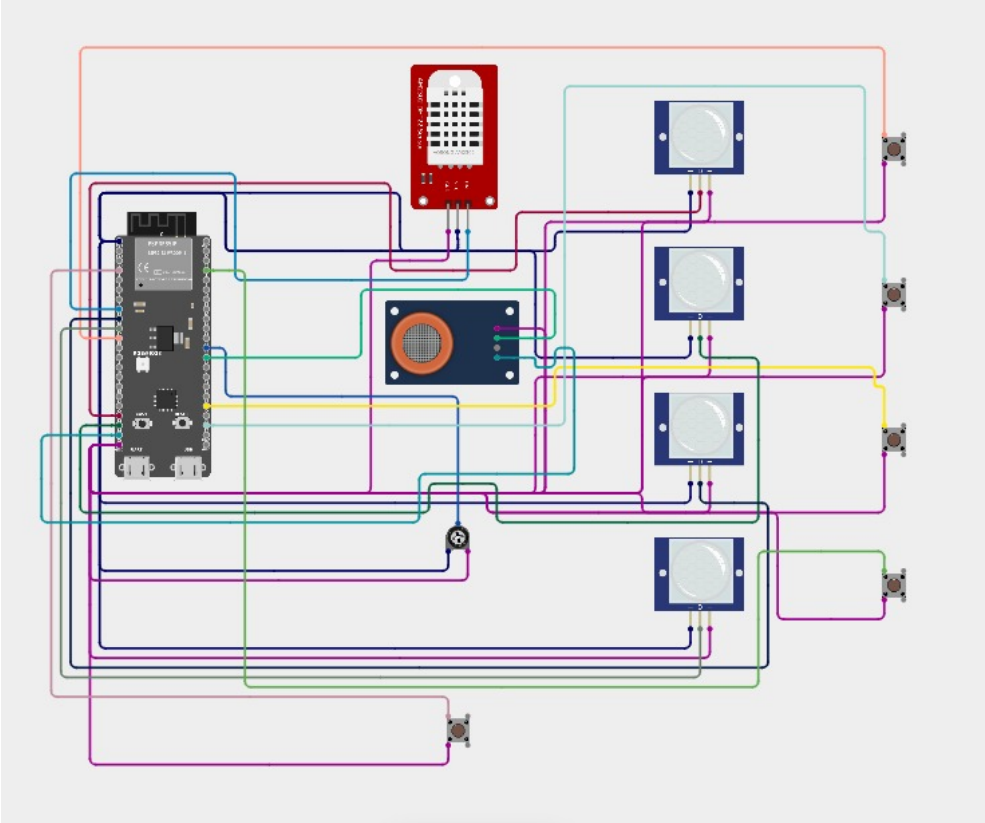


Fig. 6.2: CleanNode IoT Sensor Integration

Fig. 6.2 shows how the sensors are wired to the ESP32 microcontroller, which acts as the central controller of each node. The ESP32 collects data from the environmental, air quality, occupancy and water flow sensors and transmits it over Wi-Fi using MQTT for real-time monitoring.

The gas sensor detects ammonia and other gases that indicate a bad smell. The DHT22 measures temperature and humidity. The PIR sensors detect movement, and therefore occupancy, in each cubicle. The water flow sensor measures the flow of water through the flush line and so indicates whether a flush has taken place. Push buttons and switches are used during testing to simulate sensor and status inputs, and connecting wires carry power and signals between the components.

6.3 Database Design

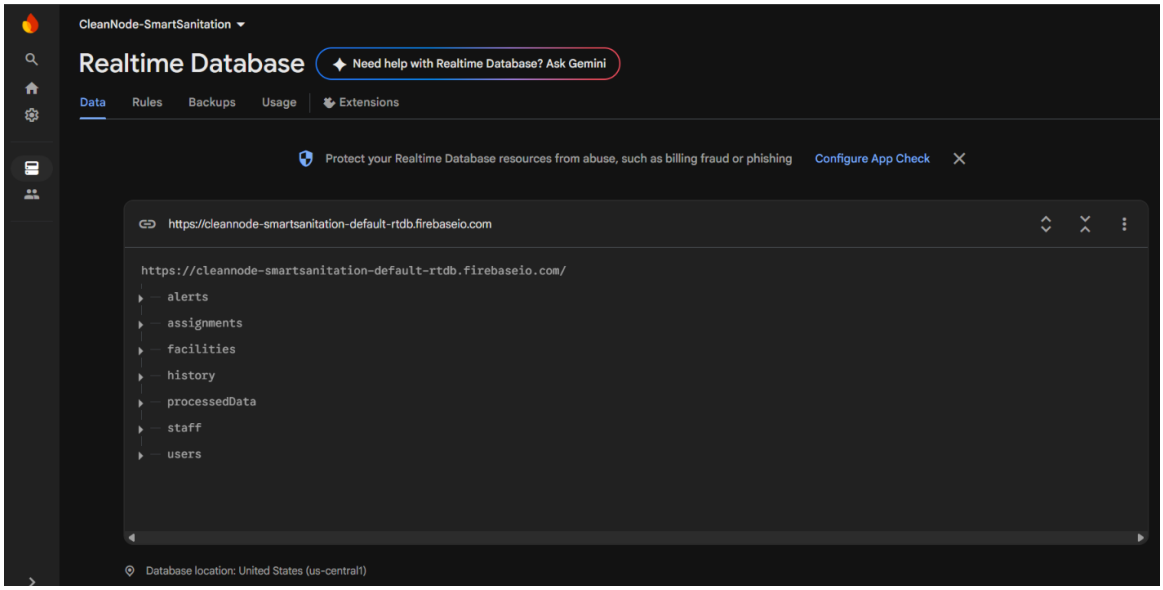


Fig. 6.3: Firebase Realtime Database Structure

CleanNode stores its data in a Firebase Realtime Database named `cleannode-smartsanitation`. Being a document-oriented store, it holds data as a tree of JSON nodes rather than as relational tables. Fig. 6.3 shows its seven top-level nodes, whose purpose is described below.

The `facilities` node holds the live sensor readings, nested by facility identifier, then by block and then by cubicle, so that a single facility can contain several cubicles. The `alerts` node holds the alerts that are currently active, such as Flush Issue, Bad Smell and Danger Zone. The `assignments` node records the cleaning duties assigned to staff. The `history` node keeps a time series of sensor readings for each area and facility. The `processedData` node stores computed values such as the PCI and other cleanliness scores. The `staff` node holds staff profiles and roles, and the `users` node maps each Firebase Authentication account to its role as Admin, Staff or Public.

6.4 Document Structure


```
{
  "alert": "Bad smell"
   ,
  "gas": 494,
  "humidity": 64,
  "occupancy": 0,
  "pci": 62.5,
  "timestamp": "
   2026-04-09
   22:54:53.902144",
  "water_flow": 0
}
```


Listing 6.1: Bad Smell


```
{
  "alert": "Normal",
  "gas": 390,
  "humidity": 46,
  "occupancy": 1,
  "pci": 61.3,
  "timestamp": "
   2026-04-09
   22:49:53.902184",
  "water_flow": 1
}
```


Listing 6.2: Normal


```
{
  "alert": "Flush
   issue",
  "gas": 245,
  "humidity": 46,
  "occupancy": 3,
  "pci": 33.55,
  "timestamp": "
   2026-04-09
   22:34:53.902240",
  "water_flow": 0
}
```


Listing 6.3: Flush Issue

Each reading sent by a sensor node is stored as one JSON document with seven fields: `gas` (air quality reading), `humidity`, `occupancy`, `water_flow`, the computed `pci` value, a `timestamp`, and an `alert` field that records the condition detected at that moment. The three listings show one document for each type of condition. In the first, a high gas reading of 494 has triggered a bad-smell alert. The second shows a normal reading taken while the restroom is in use, with water flow present. In the third, the occupancy count is three but no water flow has been recorded, so the system records a flush issue and the PCI falls to 33.55. Keeping every reading as a separate, timestamped document allows the same data to drive the live dashboard, the alerts list and the history view.

7. IMPLEMENTATION

7.1 Algorithms

This section describes the three main procedures of CleanNode: collecting and publishing sensor data at the node, computing the cleanliness index and alerts at the backend, and detecting anomalies across several sensors. Threshold values are shown as named limits because the authority can change them from the Settings page of the dashboard.

Algorithm-1: Sensor Data Acquisition and Publishing

Input: Readings from the MQ-135, DHT22, PIR and YF-S201 sensors

Output: JSON message published to the MQTT topic `pes/cleannode/iot`

Methodology:

Repeat continuously while the node is powered

Step 1: Read temperature and humidity from the DHT22 every 2 seconds.

Step 2: Read the gas level from the MQ-135 and the occupancy state from the PIR sensor.

Step 3: Count the pulses from the YF-S201 flow sensor over the interval and convert the pulse frequency F to flow rate Q using $F = 7.5\,Q$.

Step 4: Every 5 seconds, build a JSON message containing the facility and cubicle identifiers and the latest readings.

Step 5: Publish the message to the MQTT broker over Wi-Fi.

End Repeat.

Algorithm End.

Algorithm-2: Cleanliness Index (PCI) Computation and Alert Generation

Input: Sensor message received from the MQTT broker

Output: PCI value (0 to 100), cleanliness status and alerts stored in Firebase

Methodology:

For each message received do

Step 1: Extract gas level, humidity, temperature, occupancy and water flow from the message.

Step 2: Pass these values to the trained Random Forest regression model to obtain the PCI on a scale of 0 to 100.

Step 3: Write the reading and the PCI to the `facilities`, `history` and `processedData`

nodes.

Step 4: If the gas level exceeds GAS_LIMIT, mark the reading as Bad Smell. If occupancy was detected but no water flow followed, mark it as Flush Issue. If the PCI falls below the Danger Zone limit, mark it as Danger Zone.

Step 5: Raise an alert only when the same condition is found in 3 consecutive readings (3-cycle debouncing), so that a single noisy reading does not trigger a false alert.

Step 6: Write the alert to the `alerts` node and start a 15-minute timer for the facility.

Step 7: If a critical alert is still unresolved when the timer expires, escalate it on the dashboard and record it against the facility.

End For.

Algorithm End.

Algorithm-3: Multi-Sensor Anomaly Detection

Input: Latest readings and PCI for each cubicle

Output: List of anomalies with severity Critical, Moderate or Minor

Methodology:

For each cubicle do

Step 1: Check each condition: high gas level, abnormal humidity, high occupancy, occupancy without water flow, and low PCI.

Step 2: Count the number of conditions found together.

Step 3: Assign a severity of Critical, Moderate or Minor according to the number and type of conditions found.

Step 4: Add the cubicle, its conditions and its severity to the anomaly list shown on the Anomaly Detection page.

End For.

Algorithm End.

7.2 Screenshots

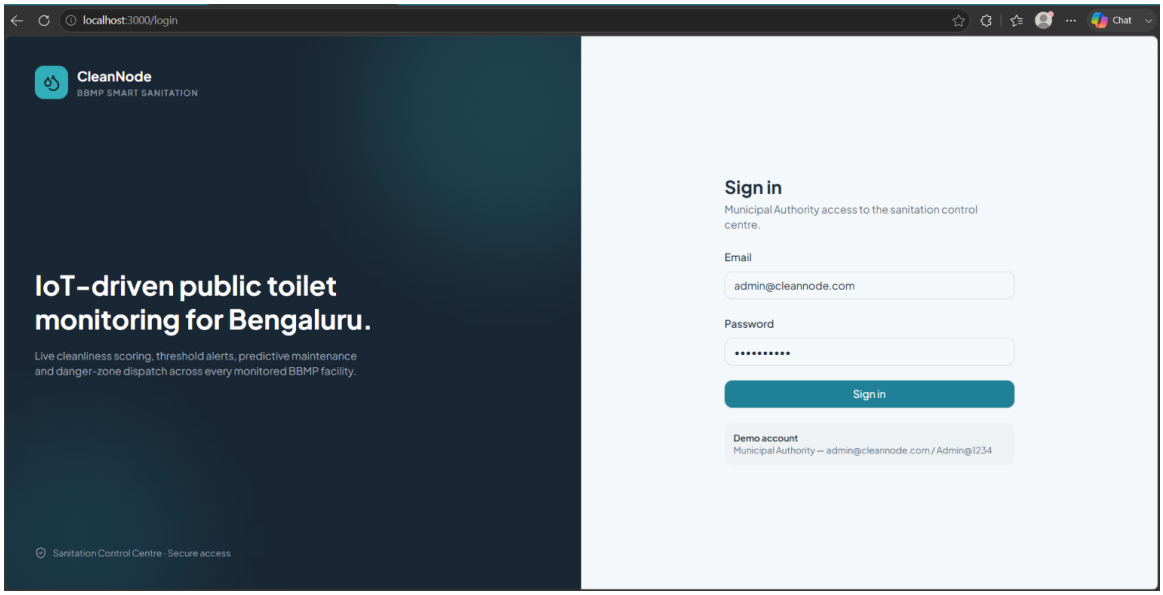


Fig. 7.1: Login Screen

The Login screen is the entry point to the CleanNode web dashboard. Municipal authority officials and maintenance staff sign in with the credentials issued to them, and Firebase Authentication verifies each user and the role attached to the account before the dashboard opens. Only authenticated users can reach the sanitation-management functions, which meets the security requirement in the SRS.

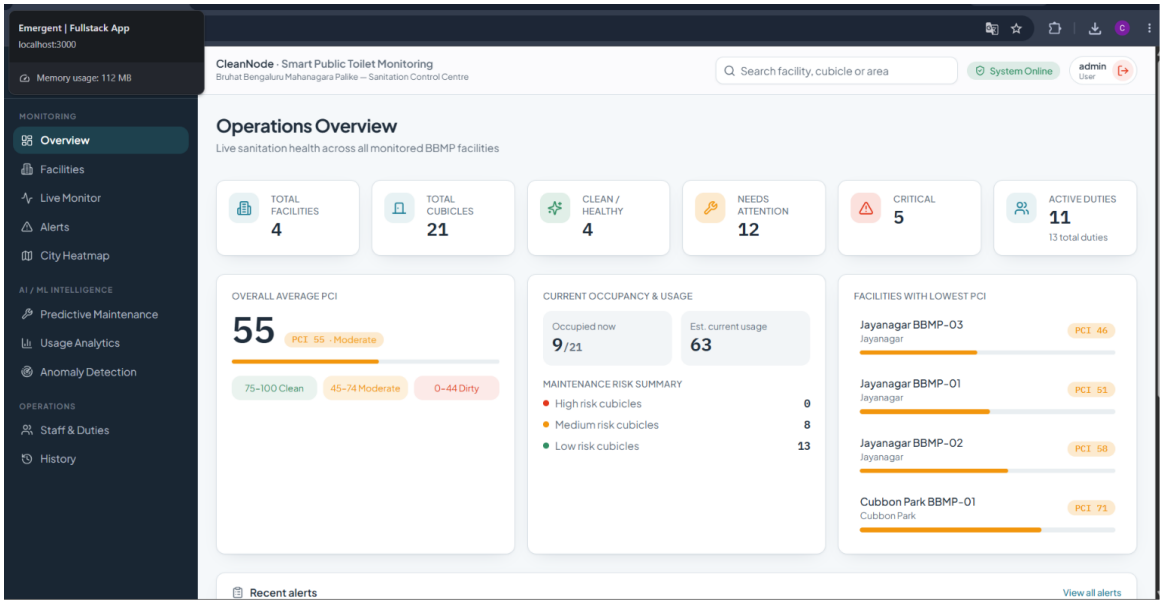


Fig. 7.2: Operations Overview

The Operations Overview gives the authority a single view of every monitored facility and cubicle. It shows the total number of facilities and cubicles, how many are clean, how many need attention and how many are critical, the number of active duties, the overall

PCI, current occupancy and usage, and a list of facilities with low PCI. An official can therefore see at a glance where staff are needed.

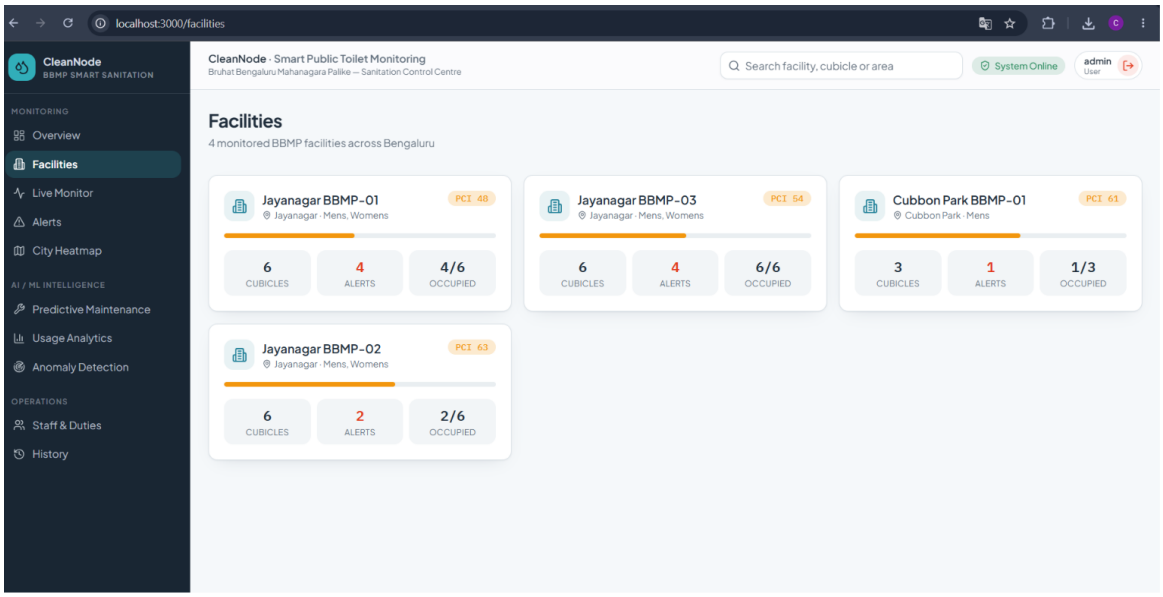


Fig. 7.3: Facilities

The Facilities page lists each monitored BBMP facility with its PCI, number of cubicles, active alerts and current occupancy, so that conditions can be compared across locations. It also shows that CleanNode is not limited to a single toilet: each facility is organised into blocks and cubicles that are monitored separately.

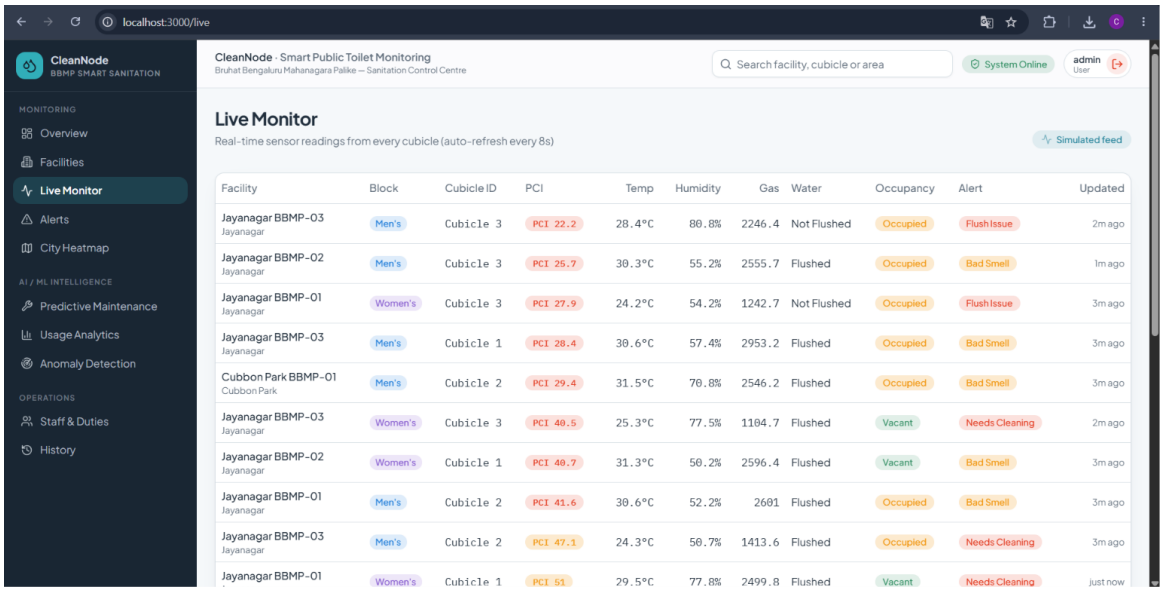


Fig. 7.4: Live Monitor

The Live Monitor shows the latest sensor readings for every cubicle: temperature, humidity, gas level, water and flush status, occupancy, PCI and any active alert. The table refreshes automatically as new readings arrive, and reflects the full path of the data from the sensor node through the backend and Firebase to the dashboard.

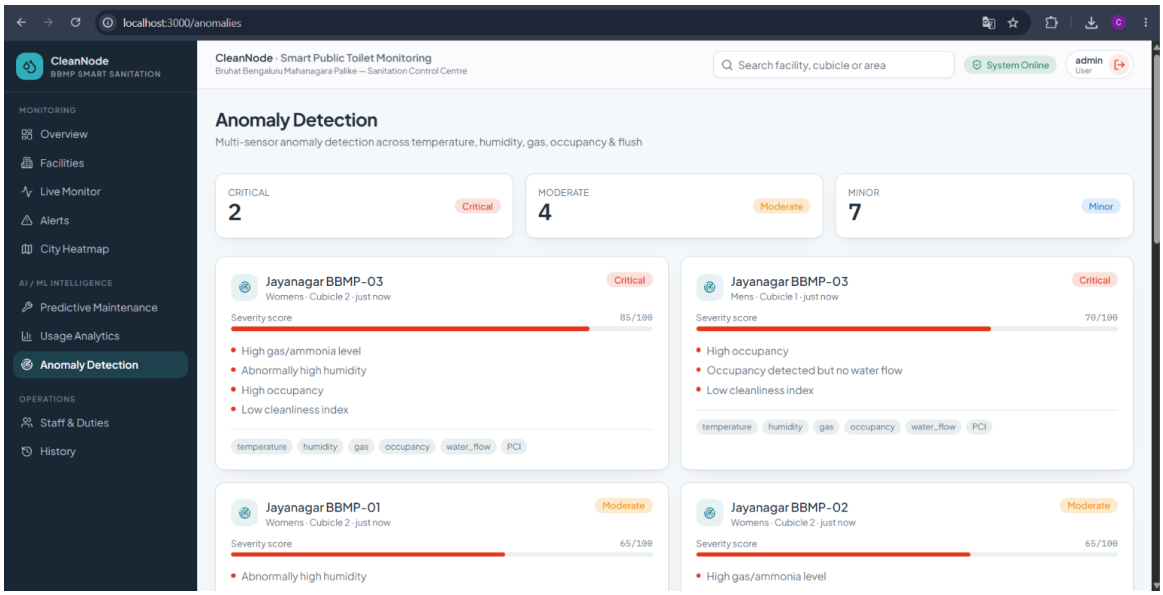


Fig. 7.5: Anomaly Detection

The Anomaly Detection page lists cubicles where an unusual combination of sensor conditions has been found and gives each a severity of Critical, Moderate or Minor. Examples include a high gas level, abnormal humidity, high occupancy, occupancy without water flow, and a low PCI. Because the assessment combines several sensors rather than relying on any one of them, this page shows the AI/ML component of the system at work.

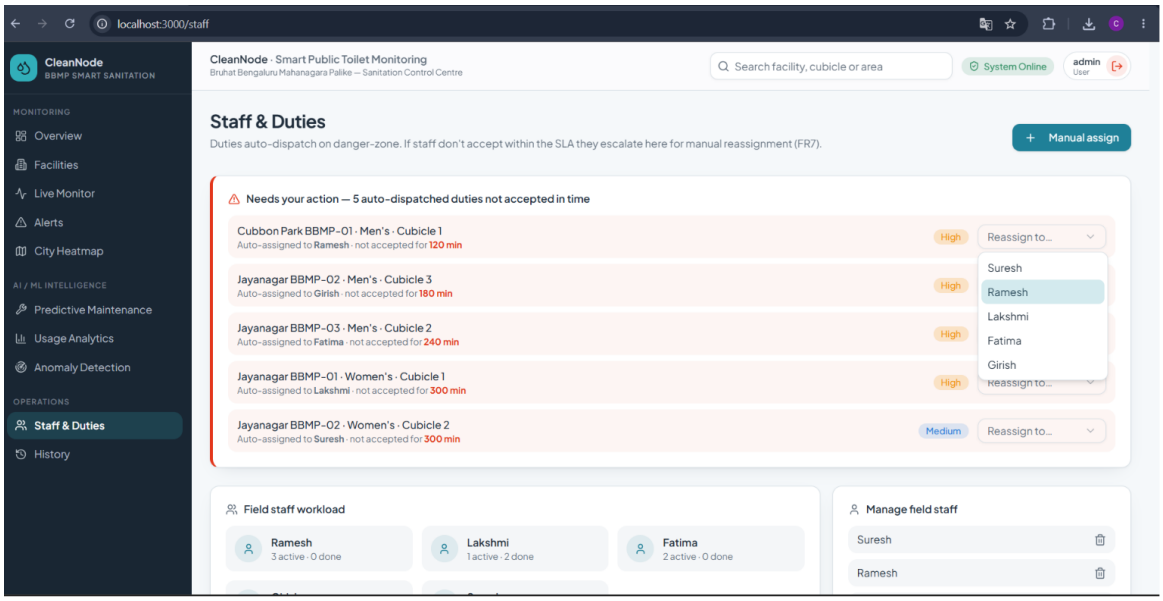


Fig. 7.6: Staff and Duties

The Staff and Duties page manages cleaning work. It shows the duties assigned to each staff member, each person's workload, the priority of each duty, and the duties that need to be reassigned. From this page the authority can assign a new duty or move an existing one to another staff member.

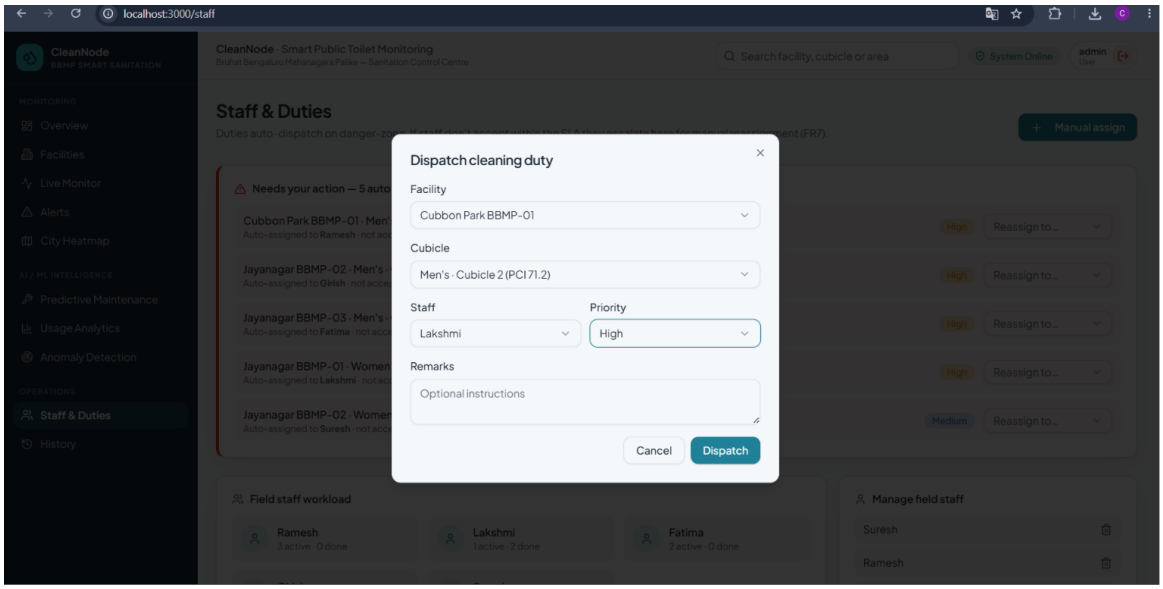


Fig. 7.7: Dispatch Cleaning Duty

The Dispatch Cleaning Duty form is used to assign a task. The authority selects the facility, the cubicle, the staff member and the priority, and can add remarks before dispatching the duty. If the original staff member cannot complete a task, the reassignment list offers the other available staff. Together these functions let maintenance follow actual conditions rather than a fixed cleaning schedule.

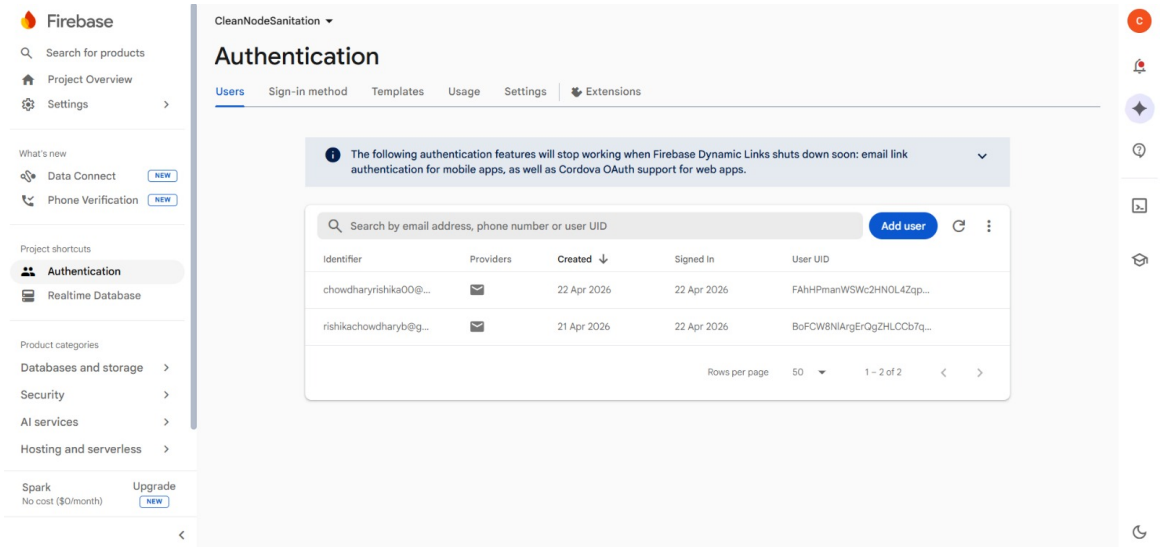


Fig. 7.8: Firebase Authentication

User accounts are managed through Firebase Authentication. Each account is identified by a unique user ID, and Firebase records its creation date and last sign-in time. The role of each account (Admin, Staff or Public) is held in the `users` node of the database and decides which screens the user can open.

8. SOFTWARE TESTING

CleanNode was tested manually at three levels: the sensor hardware, the web dashboard used by the municipal authority, and the Android application used in the field and by the public. Each test case states the condition applied, the result expected, the result actually observed and whether the test passed. Table 8.1 lists the subsystems that were tested and the technologies each one uses.

Table 8.1: Subsystems Under Test

Subsystem	Core Technologies
Hardware Edge and Sensors	ESP32 DevKit V1, MQ-135, DHT22, PIR, YF-S201
Cloud Telemetry Broker	HiveMQ Cloud MQTT broker (port 1883, topic `pes/cleannode/iot`)
AI/ML Inference and Backend	FastAPI, Python 3.12, Scikit-Learn RandomForestRegressor, Firebase Realtime Database
Admin Web Dashboard	React 18, Vite/Craco, Tailwind CSS, Lucide Icons, Sonner Notifications
Field Android Mobile App	Kotlin, Jetpack Compose, Android 14, Android Room (SQLite), CameraX

8.1 Hardware and Sensor Testing

The hardware tests check that each sensor responds correctly to the condition it is meant to detect and that the ESP32 delivers its readings to the cloud without loss. Table 8.2 lists the five hardware test cases.

Table 8.2: Test Cases: Hardware and Sensors

Test ID	Module	Test Condition	Expected Result	Observed Result	Status
TC-HW-01	Gas Sensor (MQ-135)	Sensor exposed to test smoke / foul air	Gas reading exceeds threshold ($>$ 2500 ppm)	Reading rose from 420 ppm to 2850 ppm	Pass
TC-HW-02	Motion Sensor (PIR)	Hand wave / person moving in front of sensor	Digital signal changes from LOW (0) to HIGH (1)	Occupancy status changed to "Occupied" immediately	Pass
TC-HW-03	Water Flow (YF-S201)	Water flows through turbine during flush	Interrupt pin counts pulses ($F = 7.5Q$)	Measured 3.5 litres of water dispensed	Pass
TC-HW-04	Temperature / Humidity (DHT22)	Continuous sampling of the room	Temperature and humidity read every 2 s	Recorded 26.8°C and 62% RH consistently	Pass
TC-HW-05	Edge Data Push (ESP32)	ESP32 sends data over Wi-Fi to MQTT / cloud	280-byte JSON payload sent every 5 s	Packets reached the cloud gateway with no drops	Pass

8.2 Web Dashboard Testing

The web dashboard tests cover login, switching between wards, live updates, configurable limits and escalation of unresolved issues. The five test cases are given in Table 8.3.

Table 8.3: Test Cases: Web Dashboard

Test ID	Module	Test Condition	Expected Result	Observed Result	Status
TC-WEB-01	Admin Login	Log in through Firebase Authentication	Admin is verified and allowed to log in	Role verified and login succeeded	Pass
TC-WEB-02	Ward Filtering	Switch from ward 153 to ward 111	Data changes to the selected ward without a page refresh	Ward changed instantly	Pass
TC-WEB-03	Live Sensor Update	Change a sensor value in Firebase	Dashboard shows the new value immediately	New value appeared automatically	Pass
TC-WEB-04	Settings	Change the cleanliness limit in Settings	Toilet status changes according to the new limit	Status updated without restarting the system	Pass
TC-WEB-05	Cleaning Alert	Leave a critical cleaning issue unresolved for 15 minutes	System shows an alert	Alert appeared and the issue was recorded	Pass

8.3 Mobile Application Testing

The mobile application tests check that the Android app shows live figures, reacts to a simulated fault, displays alerts and the restroom map, and saves feedback from citizens. Table 8.4 lists these test cases.

Table 8.4: Test Cases: Mobile Application

Test ID	Module	Test Condition	Expected Result	Observed Result	Status
TC-MOB-01	Live KPI Overview	Launch the app connected to Firebase	Real-time KPI counts shown (5 monitored, average PCI 75/100)	All four cards filled with live ward figures	Pass
TC-MOB-02	Live Spike Demo	Tap “Trigger Spike” on the Overview screen	Simulated gas spike drops the PCI into the Danger Zone	Danger Zone count rose to 1 and status turned Critical	Pass
TC-MOB-03	Real-Time Alerts	Open the Alerts tab after the spike	Critical alert shown with timestamp and cubicle ID	Alert displayed immediately for Jayanagar e-Toilet	Pass
TC-MOB-04	Restroom Map	Open the Map tab	BBMP public toilet locations in Bengaluru shown on the map	Markers displayed with ward names	Pass
TC-MOB-05	Citizen Feedback	Submit a 4/5 star rating with a comment	Feedback record saved to the central database	Confirmation shown and feedback stored in Firebase	Pass

All fifteen test cases across the hardware, web dashboard and mobile application passed.

9. CONCLUSION

This project set out to replace the fixed cleaning schedules and manual inspection used for public toilets with maintenance driven by the actual condition of each facility. CleanNode achieves this with a sensor node in each cubicle, built on an ESP32 with MQ-135, DHT22, PIR and YF-S201 sensors, which streams its readings over MQTT to a cloud backend. There, a Random Forest model computes a Predictive Cleanliness Index (PCI) between 0 and 100, using three-cycle debouncing so that a single faulty reading does not raise a false alarm.

The results are delivered through two interfaces. The React web dashboard gives the municipal authority an overview of every facility and cubicle, live readings, anomaly detection and tools to assign and reassign cleaning duties. The Android application supports staff in the field and lets members of the public find toilets on a map and submit feedback. When a severe gas or flush problem is detected, the system starts a 15-minute countdown and escalates the issue if it has not been resolved in that time, which gives the authority a clear record of response times.

The system was verified through fifteen manual test cases covering the sensor hardware, the web dashboard and the mobile application, all of which passed. The main contributions of the project are the combination of several sensor readings into a single cleanliness score, the detection of flush failures by relating occupancy to water flow, and a workflow that links an alert directly to the assignment of a cleaning duty. Together these make CleanNode a practical basis for condition-based maintenance of public toilets in a city such as Bengaluru.

10. FUTURE ENHANCEMENTS

Several extensions would make CleanNode more autonomous and more robust in the field.

Automated flushing and sanitiser spraying. Electronic solenoid valves could be added so that a disinfectant spray or a high-pressure flush is triggered automatically when a sharp rise in ammonia is detected, without waiting for a cleaner to arrive.

Solar power. Rooftop solar panels with LiFePO4 battery modules would make each sensor node self-powered and keep it running during cuts in the municipal power supply.

Monitoring of consumables. Ultrasonic level sensors in overhead water tanks and liquid soap dispensers would allow the system to predict when they need refilling and prevent water or soap from running out.

Fault and tamper detection. Diagnostic checks could identify a disconnected wire, a covered sensor or physical damage to a node, and alert administrators to the fault immediately.

Location-based alerts for staff. Using GPS geofencing on the mobile application, a sanitation worker who comes within 100 metres of a restroom in the Danger Zone could receive an immediate alert on their phone.

APPENDIX A: BIBLIOGRAPHY

Research Papers

[1] F. I. Azman, N. L. Saleh, F. Hashim, A. Sali, A. M. Ali and A. S. M. Noor, An IoT-Based Hygiene Monitoring System in the Restroom, *IEEE Access*, Vol. 13, 2025, pp. 119348–119361, doi: 10.1109/ACCESS.2025.3586117.

[2] P. Deshmukh, A. Mohite, H. Bhoir, R. Patil and A. Bhonde, Intelligent Public Toilet Monitoring System Using IoT, *Proc. of IEEE Bangalore Humanitarian Technology Conference (B-HTC)*, Oct. 2020, doi: 10.1109/B-HTC50970.2020.9297839.

[3] K. Dineva and T. Atanasova, Design of Scalable IoT Architecture Based on AWS for Smart Livestock, *Animals*, Vol. 11, No. 9, Sep. 2021, Art. No. 2697, doi: 10.3390/ani11092697.

[4] A. Lokman, R. K. Ramasamy and C.-Y. Ting, Scheduling and Predictive Maintenance for Smart Toilet, *IEEE Access*, Vol. 11, 2023, pp. 17983–17999, doi: 10.1109/ACCESS.2023.3241942.

[5] T. Stephan, S. M. Hari Krishna, C.-C. Lin, U. Sumesh, S. Agarwal and H. Kim, ProWaste for Proactive Urban Waste Management Using IoT and Machine Learning, *Scientific Reports*, Vol. 15, Jul. 2025, doi: 10.1038/s41598-025-08452-x.

[6] K. Pardini, J. J. P. C. Rodrigues, O. Diallo, A. K. Das, V. H. C. de Albuquerque and S. A. Kozlov, A Smart Waste Management Solution Geared Towards Citizens, *Sensors*, Vol. 20, No. 8, Apr. 2020, Art. No. 2380, doi: 10.3390/s20082380.

[7] M. Vibhute, J. Biju, K. Shelar, N. Koli and O. Gubyad, Smart Drainage Monitoring System, *International Journal for Multidisciplinary Research (IJFMR)*, Vol. 7, No. 2, Apr. 2025, doi: 10.36948/ijfmr.2025.v07i02.40325.

[8] Q. P. Ha, S. Metia and M. D. Phung, Sensing Data Fusion for Enhanced Indoor Air Quality Monitoring, *IEEE Sensors Journal*, Vol. 20, No. 8, Apr. 2020, pp. 4430–4441, doi: 10.1109/JSEN.2020.2964396.

[9] H. Zhang, R. Zhang and J. Sun, Developing Real-Time IoT-Based Public Safety Alert and Emergency Response Systems, *Scientific Reports*, Vol. 15, Aug. 2025, doi: 10.1038/s41598-025-13465-7.

[10] M. Romano, P. Díaz and I. Aedo, Gamification-less: May Gamification Really Foster Civic Participation? A Controlled Field Experiment, *Journal of Ambient Intelligence and Humanized Computing*, Jun. 2021, doi: 10.1007/s12652-021-03322-6.